COS10022 Data Science Principles — Assignment 1

Predictive Model Creation and Evaluation

Name: [your full name]
Student ID: [student ID]
Unit code: COS10022
Class: [CL1/0x]
Tutor: [tutor's name]

1 Introduction

This report documents a KNIME Analytics Platform (5.12) workflow that predicts whether a person survived a road accident from five attributes: Age, Gender, Speed_of_Impact, Helmet_Used and Seatbelt_Used. Task 1 builds a logistic regression on a specified training and test file. Task 2 combines all CSV files, builds a balanced training set and a separate test set, and compares logistic regression, Naïve Bayes and Random Forest on identical data. Both tasks are in one workflow, `COS10022_A1_Workflow`. The input files are stored inside the workflow and every file path is relative to it, so the workflow runs unchanged on any computer. Every random step uses the fixed seed 2026, so re-execution reproduces the results in this report.

1.1 Dataset and data exploration

The data is clean: no missing values, numeric attributes within plausible ranges (Age 18–69, Speed 20–119) and consistent Yes/No and Male/Female categories. Exploring the files before modelling revealed four properties that shaped the workflow (Table 1):

1. **The four CSV files are two different 70/30 splits of the same 600 records**: train1+test1 and train2+test2 contain the same rows. Combining all four gives 1,200 rows but only 599 distinct records.
2. **Every file is sorted**: all "No" rows first, then all "Yes", each in descending speed.
3. **The classes are imbalanced**: 378 Yes (31.5%) and 822 No across the four CSVs.
4. **Speed acts as a threshold**: none of the 191 people involved at 90 km/h or more survived, while 46.2% of the 409 below 90 survived. Below 90, wearing a helmet (54.8% vs. 37.2%) or a seatbelt (57.6% vs. 35.0%) is associated with higher survival.

Table 1: Input files (folder `src`, unchanged)

File	Rows	Format	Used in
`accident_2025_train1_73.csv`	420	comma-separated	Task 2
`accident_2025_train2_73.csv`	420	comma-separated	Task 1 (training), Task 2
`accident_2025_test1_73.csv`	180	comma-separated	Task 2
`accident_2025_test2_73.csv`	180	comma-separated	Task 2
`accident_2025_test2_73.txt`	180	**tab-separated**, CRLF	Task 1 (test)

Cleaning decisions. No cleaning transformations were applied, and each was considered deliberately. Normalisation was unnecessary: the IRLS solver used for logistic regression does not need scaled inputs to converge, Naïve Bayes estimates a distribution per attribute, and Random Forest compares split thresholds, so none of the three is affected by attribute scale. Duplicates were *not* removed, because only 189 distinct survivors exist and the brief requires 350; their effect on evaluation is measured instead (Section 4). Survived was kept nominal, as the classifiers require, and all five attributes were used, as the brief specifies.

2 Task 1: Logistic Regression

2.1 Workflow design

The Task 1 branch (Figure 1) follows the model-building phase of the Data Analytics Lifecycle: read the training data, fit the model, apply it to separate test data, and evaluate. The two readers are configured and checked independently, so a loading error cannot reach the model unnoticed. The learner receives only training data, and the predictor receives the fitted model plus the test data. A CSV Writer saves the predictions as an output file.

SCREENSHOT NEEDED
Task 1 part of the canvas: nodes CSV Reader x2, LR Learner, LR Predictor, Scorer, CSV Writer (zoom in until the node labels are readable)
save as `figures/t1_workflow.png`

Figure 1: Task 1 workflow

2.2 Node configuration

Table 2: Task 1 node configuration

Node	Configuration	Reason
CSV Reader [1]	`src/accident_2025_train2_73.csv`; comma; header row; relative to workflow data area	training file named in the brief; a relative path keeps the workflow portable
CSV Reader [2]	`src/accident_2025_test2_73.txt`; **Tab** delimiter; header row	the file is tab-separated (Section 2.3)
Logistic Regression Learner [3]	target Survived; all five attributes; reference category **No**; solver **IRLS**; max. 100 epochs, $\epsilon = 10^{-5}$; coefficient statistics on	with reference "No", coefficients describe the odds of surviving. IRLS (iteratively reweighted least squares) gives the exact maximum-likelihood solution deterministically; the stochastic SAG solver is approximate and suited to large data
Logistic Regression Predictor [4]	model from [3], data from [2]; append class probabilities	probabilities show the model's confidence, not only its label
Scorer [5]	actual Survived vs. Prediction (Survived)	confusion matrix and accuracy statistics
CSV Writer [6]	`output/task1_LR_predictions.csv`; overwrite if exists	output file required for the submission; overwrite lets the workflow re-run without failing

SCREENSHOT NEEDED
Logistic Regression Learner [3] configuration dialog: target column, reference category = No, solver = IRLS, included columns
save as `figures/t1_lr_learner_config.png`

Figure 2: Logistic Regression Learner configuration

2.3 Difficulties loading the files and how they were solved

The training file loaded correctly with default settings, but the test file did not. With the default comma delimiter, the CSV Reader placed each line of `accident_2025_test2_73.txt` into a single column (Figure 3, left). The predictor could therefore not find Age, Speed_of_Impact or the other attributes the model was trained on. Opening the file showed the cause: unlike every other file, it separates values with **tabs** and ends lines with Windows (CRLF) line breaks. Setting the column delimiter to Tab (`\t`) fixed it; the line-break row delimiter handles CRLF. The fix was confirmed by checking that the test table has 180 rows and the same six columns, with the same names and types (integer Age and Speed, string attributes), as the training table. Matching columns are a requirement for applying a trained model. The identical `test2.csv` was not used instead, because the brief names the `.txt` file.

SCREENSHOT NEEDED
CSV Reader [2] preview with the default comma delimiter: everything in ONE column
save as `figures/t1_loading_before.png`

SCREENSHOT NEEDED
CSV Reader [2] with Tab delimiter: 6 separate columns, 180 rows
save as `figures/t1_loading_after.png`

Figure 3: Test file read with the default comma delimiter (left) and with the Tab delimiter (right)

2.4 Prediction results

Table 3: Task 1 results on the 180-row test set (positive class = Yes)

	Predicted No	Predicted Yes
Actual No (111)	92	19
Actual Yes (69)	45	24

Accuracy	0.644
Always-“No” baseline	0.617
Cohen’s kappa	0.190
Recall / Precision (Yes)	0.348 / 0.558
Specificity	0.829

SCREENSHOT NEEDED
Scorer [5] output: confusion matrix and the accuracy statistics table
save as `figures/t1_scorer.png`

Figure 4: Task 1 Scorer output

The model is correct for 64.4% of the test cases, only 2.7 percentage points better than always predicting “No” (61.7%). It recognises most non-survivors (specificity 0.83) but finds only 35% of the

survivors: 45 of the 69 survivors are predicted to die. This lean towards "No" follows from the training data, where 71% of the rows are "No". Kappa (0.19) confirms only slight agreement beyond chance.

Table 4: Logistic regression coefficients (reference category No; base levels Female / No)

Attribute	Coefficient	Std. error	p-value	Odds ratio
Intercept	0.285	0.492	0.562	1.33
Age	−0.001	0.009	0.910	1.00
Gender = Male	0.407	0.249	0.103	1.50
Speed_of_Impact	−0.035	0.005	< 0.001	0.97
Helmet_Used = Yes	0.808	0.250	0.001	2.24
Seatbelt_Used = Yes	1.008	0.252	< 0.001	2.74

SCREENSHOT NEEDED
Logistic Regression Learner [3] output port "Coefficients and Statistics" (right-click the node), to confirm Table 4
save as `figures/t1_coefficients.png`

Figure 5: Coefficients and statistics output of the Logistic Regression Learner

The coefficients (Table 4) agree with domain expectations. Each extra km/h multiplies the odds of surviving by 0.965, so 10 km/h faster reduces the odds by about 30%. Wearing a seatbelt multiplies the odds by 2.74 and a helmet by 2.24; both effects are significant ($p \leq 0.001$). Age and Gender are not significant at the 5% level. The model's weakness is structural: logistic regression fits a smooth linear trend in the log-odds, while the data contains a hard threshold (no survivors at 90 km/h or more), which a linear boundary can only approximate.

3 Task 2: Data preparation and model comparison

3.1 Workflow design

Task 2 (Figure 6) has two stages. Data preparation reads all CSV files, separates the classes, draws exactly 350 records of each class into the training set and puts the remainder into the test set. Modelling then feeds the *same* shuffled training table to all three learners and the *same* shuffled test table to all three predictors, so any difference in results comes from the algorithms, not from the data. Test data never reaches a learner. CSV Writers save both sets and every model's predictions.

SCREENSHOT NEEDED
Task 2 part of the canvas: CSV Reader [10] -> Rule-based Row Splitter -> 2x Table Partitioner -> 2x Concatenate -> 2x Shuffle -> LR / NB / RF learners, predictors, scorers
save as `figures/t2_workflow.png`

Figure 6: Task 2 workflow

3.2 Taking in all CSV files

One CSV Reader [10] in *Files in folder* mode reads the `src` folder with a file-extension filter of `csv`. This includes the four CSV files and excludes the `.txt`, without modifying the folder, as the brief requires (Figure 7). Three further options were set deliberately:

- *Prepend file index to RowID*: each file numbers its rows from Row0, so without a prefix the RowIDs of the four files would collide.
- *Append path column* off: otherwise a "Path" column would be added, and the learners would use the source file name as a predictor.
- *Fail on differing specs* on: a file with a different structure stops the node instead of producing corrupted rows.

The output has 1,200 rows and the six original columns.

SCREENSHOT NEEDED
CSV Reader [10] dialog: mode "Files in folder", folder src, file extension filter csv, and the advanced options (path column off)
save as `figures/t2_reader_folder_config.png`

Figure 7: Reading every CSV file in `src`

3.3 Rule-based Row Splitter

The Rule-based Row Splitter [11] uses the single rule `$Survived$ = "Yes" => TRUE`. `$Survived$` refers to the column, `"Yes"` is a text constant and `=>` gives the outcome. Rows for which the rule is TRUE go to the top output (378 survivors) and all others to the bottom output (822 non-survivors). The counts add up to 1,200, confirming no row was lost. Splitting by class first is what allows exactly 350 records to be drawn from *each* class; one random split of the whole table would produce an unpredictable class mix. Each output is still in file order (sorted by speed), which made random sampling essential in the next step. The rule also has to match the label exactly: `"yes"` would send every row to the bottom output, so the 378/822 counts were checked.

SCREENSHOT NEEDED
Rule-based Row Splitter [11] dialog showing the rule $Survived$ = "Yes" => TRUE
save as `figures/t2_splitter_rule.png`

Figure 8: Rule-based Row Splitter configuration

3.4 Workflow Coach and node selection

The Workflow Coach was used at the top output of the Rule-based Row Splitter with the prompt: [type the exact prompt you used]. It suggested [list the suggested nodes] (Figure 9). The **Table Partitioner** (called Partitioning in earlier KNIME versions) was chosen because it splits one table into two outputs by an absolute row count, keeping the remainder in the second output. Row Sampling would discard the remainder, and a rule-based splitter cannot split by count.

SCREENSHOT NEEDED

Workflow Coach / K-AI at the Rule-based Row Splitter output: your prompt and the list of suggested nodes

save as `figures/t2_workflow_coach.png`

Figure 9: Workflow Coach prompt and suggestions

3.5 Data preparation configuration

Table 5: Task 2 data-preparation configuration and resulting row counts

Node	Configuration	Reason / result
Table Partitioner [12] (Yes)	**Absolute, 350**; **draw randomly**; seed 2026	exactly 350, as required; random because the input is sorted by speed ("take from top" would select the 350 fastest crashes). Top 350, bottom 28
Table Partitioner [13] (No)	same configuration	top 350, bottom 472
Concatenate [14]	top outputs of [12] and [13]; union of columns; duplicate RowIDs get a suffix	training set: 700 rows, 350 Yes / 350 No
Concatenate [15]	bottom outputs of [12] and [13]	test set: 500 rows, 28 Yes / 472 No
Shuffle [16], [17]	seed 2026	removes the class-sorted order of both sets; the seed keeps the order reproducible
CSV Writer [18], [19]	`output/task2_train_700.csv`, `output/task2_test_500.csv`	evidence of the exact counts; intermediate files for the submission

SCREENSHOT NEEDED

Table Partitioner [12] dialog: Absolute = 350, Draw randomly, random seed 2026

save as `figures/t2_partitioner_config.png`

Figure 10: Table Partitioner configuration

3.6 Model construction

Table 6: Model configuration (all learners use training set [16]; all predictors use test set [17])

Model	Configuration	Reason
Logistic Regression [20–22]	identical to Task 1 (reference No, IRLS)	allows a direct comparison with Task 1
Naïve Bayes [23–25]	class column Survived; Gaussian per class for Age and Speed; min. std. dev. 10^{-4}; max. 20 nominal values; class probabilities appended	applies Bayes' theorem assuming attributes are independent given the class; the minimum standard deviation avoids division by zero
Random Forest [26–28]	100 trees; Information Gain Ratio; $\sqrt{k}$ attributes per split; bootstrap sampling; unlimited depth; seed 2026; confidences appended	ensemble of decision trees with majority vote. Trees split on thresholds, which suits the speed threshold. The seed makes the forest reproducible

SCREENSHOT NEEDED
Random Forest Learner [26] dialog: target Survived, 100 models, split criterion, static random seed 2026
save as `figures/t2_rf_learner_config.png`

Figure 11: Random Forest Learner configuration

3.7 Issues encountered and solutions

- **Exact counts.** A percentage split cannot guarantee exactly 350 rows, so the Table Partitioner was set to an absolute count, and the 700/500 totals were verified in the saved output files.
- **Reproducibility.** The partitioners, shuffles and Random Forest are random. Without fixed seeds every execution would produce different sets and results, and the re-run by the marker would not match this report. A seed of 2026 was set on all five random nodes, and re-executing the workflow reproduced identical outputs.
- **Reading the folder.** Without the extension filter the folder reader would include the tab-separated `.txt`, and without the RowID prefix the four files' RowIDs would collide. Both were handled in node [10].
- **Unexpected test overlap.** While interpreting the results, the training and test sets were found to share identical records. The cause is the duplicated source data (Section 4). It could not be removed without breaking the 350-per-class requirement, so it was measured and taken into account in the comparison.

3.8 Results

Table 7: Task 2 results on the 500-row test set (positive class = Yes)

Model	TN / FP / FN / TP	Accuracy	Kappa	Recall Yes	Prec. Yes	F_1 Yes	Specificity
Always "No"	472 / 0 / 28 / 0	**0.944**	0.000	0.000	–	–	1.000
Logistic Regression	333 / 139 / 8 / 20	0.706	0.131	0.714	0.126	0.214	0.706
Naïve Bayes	316 / 156 / 7 / 21	0.674	0.120	0.750	0.119	0.205	0.669
Random Forest	350 / 122 / 2 / 26	**0.752**	**0.222**	**0.929**	**0.176**	**0.295**	**0.742**

SCREENSHOT NEEDED	SCREENSHOT NEEDED	SCREENSHOT NEEDED
Scorer [22] (LR): confusion matrix + accuracy statistics save as `figures/t2_scorer_lr.png`	Scorer [25] (NB): confusion matrix + accuracy statistics save as `figures/t2_scorer_nb.png`	Scorer [28] (RF): confusion matrix + accuracy statistics save as `figures/t2_scorer_rf.png`

Figure 12: Scorer outputs for logistic regression, Naïve Bayes and Random Forest

3.9 Comparison and insights

Accuracy is the wrong yardstick here. All three models are less accurate than always predicting "No" (94.4%), because the test set is 94.4% non-survivors. Yet the baseline finds no survivor at all, while the models find 71–93% of them. Kappa and the Yes-class recall, precision and F_1 are therefore used to compare the models, and which error matters more depends on the application. Missing a survivor (Type II error) and wrongly predicting survival (Type I error) carry very different costs.

The balanced training set pushes every model towards "Yes". The models were trained on a 50/50 class mix but tested on a 94/6 mix, so they over-predict survival: at least 82% of their "Yes" predictions are wrong (precision 0.12–0.18). Naïve Bayes is affected directly, because its prior $P(\text{Yes}) = 0.5$ is far from the test rate of 0.056. The same effect appears when logistic regression is compared across the two tasks. With the same algorithm and settings, recall rises from 0.35 (Task 1, trained on 29% survivors) to 0.71 (Task 2, trained on 50%), while precision falls from 0.56 to 0.13. Only the class balance of the training data changed.

Random Forest performs best on every measure, for two reasons. First, it matches the structure of the data: a tree can split directly at a speed threshold. Logistic regression can only fit a smooth slope, and Naïve Bayes models speed as one bell curve per class. Second, it benefits from leakage. 222 of the 500 test records (44%) are identical to a training record. Random Forest scores 0.793 on those records but 0.719 on unseen ones, while logistic regression (0.698 vs. 0.712) and Naïve Bayes (0.662 vs. 0.683) show no such gap, because they cannot memorise individual records. Part of Random Forest's lead is therefore memorisation. On genuinely unseen records it performs roughly on a par with logistic regression.

The "IID" training set, examined critically. The brief describes the 700-row set as independent and identically distributed. It is balanced by design, so its class distribution matches neither the source data (31.5% survivors) nor the test set (5.6%), and 44% of the test set duplicates it. This design is useful for learning what distinguishes the classes, but its test scores overstate performance on new, real-world data.

Suggestions.

- Remove duplicates *before* splitting. With 189 distinct survivors, a stratified 70/30 split would replace the fixed 350 per class.
- Use k-fold cross-validation (X-Partitioner and X-Aggregator) instead of a single holdout split.
- Report kappa or F_1 rather than accuracy.
- Adjust the decision threshold, or correct the priors, to the real survival rate.
- Limit Random Forest tree depth to reduce memorisation.

4 Exploration and reflection

Beyond the required steps, four questions were investigated, using a verification script that reads the workflow's output files:

1. **Is the data really clean and independent?** Counting distinct records showed 599 distinct

rows out of 1,200, with only 189 distinct survivors. The task cannot be performed on de-duplicated data, so the assignment's own design creates train/test overlap.

2. **How large is the overlap, and who benefits?** Matching each test record against the training set found 222 overlapping records. Splitting each model's accuracy into seen and unseen records showed that only the model able to memorise (Random Forest) gains from the overlap.
3. **Which attribute drives survival?** Survival by speed band showed the 90 km/h threshold. This explained why a threshold-based model outperforms a linear one, linking data understanding to model choice.
4. **Why does the same algorithm behave differently?** Comparing logistic regression across the two tasks isolated the effect of the training class balance on recall and precision.

Reflection. [In your own words (4–6 sentences): what surprised you, which theory explained it, what you learned about KNIME and about evaluating models, and what you would do differently next time.]

5 Conclusion

The workflow loads both file formats correctly, prepares exactly the training and test sets the brief specifies, and evaluates three classifiers reproducibly on identical data. Logistic regression in Task 1 reached 64.4% accuracy, barely above the majority baseline. Random Forest gave the best Task 2 results (accuracy 0.752, kappa 0.222, recall 0.929), helped by the speed threshold and partly by train/test overlap. The main lessons are that accuracy misleads on imbalanced data, that the class balance of the training set shifts a model's predictions, and that duplicated records must be detected before data is split.

References

- Dietrich, D. (ed.) 2015, *Data Science & Big Data Analytics: Discovering, Analyzing, Visualizing and Presenting Data*, EMC Education Services, Wiley, Indianapolis.
- KNIME AG 2026, *KNIME Analytics Platform 5.12 node documentation*, KNIME Hub, viewed [date], `https://hub.knime.com`.
- Swinburne University of Technology 2026, *COS10022 Data Science Principles lecture notes, weeks 1–4*.

A Generative AI declaration

Tool	Used for	Prompts / outputs, and what I verified or changed
Claude (Anthropic), via Claude Code	Reading the brief and data files; finding data issues (tab delimiter, duplicated records, sorting, imbalance); building the KNIME workflow configuration; writing Python scripts that verify the workflow outputs; explanatory notes; drafting this report	Prompts: requests to analyse the assignment, build and finalise the workflow, explain the design, and draft the report with screenshot placeholders. [add the exact prompts and outputs required by your tutor, and describe what you checked, changed and rewrote yourself]
KNIME Workflow Coach / K-AI	Finding the node to partition the data (part of the task)	[your prompt and the suggestions]